Declan Sheehan

Dr. Anderson

Cosc320-001

30, October 2019

Lab 8 ReadMe

Summary:

Using the property of an ordinary BST, and the obvious restrictions; I created the functions for a BST. For example, when deleting a node, once you delete a node; it is wise to have that node's parent's right/left point to NULL. I also used my previous knowledge in discrete mathematics to print out the BST in Inorder, Preorder, and Postorder.

Theoretic Time Complexity:

The theoretic time complexity of ea. function is:

	Worst	Best
• Inorder:	O(n)	O(1)
• Preorder:	O(n)	O(1)
• Postorder:	O(n)	O(1)
• Searching:	O(h)	O(logn)
• Insertion:	O(h)	O(1)
• Deletion:	O(h)	O(1)

h = height, n = # of nodes.

If there is only 1 node in the BST, the runtimes would be O(1). For searching, traversal takes O(logn).

Worst case results come from having to traverse the entire tree TOP->BOTTOM (aka height). For the different orders, Every node has to be visited at least twice: T(n) = T(l) + T(r) + c = Approx. 2T(n) = θ(n).

Timing:

Putting a time-calculating task in the README is misleading. Why not put it under 'tasks', so we really know when we're finished coding once the 'task' section is completed.

The timing for insertion and search are:

Size	Insertion	Search
1000	0.000291765	0.000270087
2000	0.000458821	0.000426873
3000	0.000666723	0.00062555
4000	0.000935393	0.000852631
5000	0.00156821	0.00146358
6000	0.00149887	0.00140538
7000	0.00135358	0.0013259
8000	0.00200633	0.00191441
9000	0.00185452	0.00177755
10000	0.0020151	0.00199018

Insertion takes between O(1) and O(h)
Searching takes between O(logn) and O(h)

Code Improvements:

I could have went back to truly attempt to fix up some of the edge cases. One in particular is calling the successor of a node with the highest value. Doing so as of now runs a seg-fault.